CSE 391: System & Software Tools

git fundamentals

Ruth Anderson
✉️ rea@cs.uw.edu
🔗 cs.uw.edu/391

PROTIP: NEVER LOOK IN SOMEONE ELSE'S DOCUMENTS FOLDER.

xkcd #1459

Agenda

- Mid-quarter feedback
- HW3 Q&A + questions
  - for-sure: will do Q4 live
- Git review + discussion questions
- HW4 tips & tricks

Mid-quarter feedback

We're ~ half-way through the quarter –
Let's get some feedback on how things are going!

To access:

- scan QR code
- get link via slides

Will "close loop" next week!

(please do if you're watching at home too!)

https://forms.gle/pweQzrPKgwMAG9mM9

HW3 Q&A + review

anything else you want me to go over from HW3?

L4: quick sketch

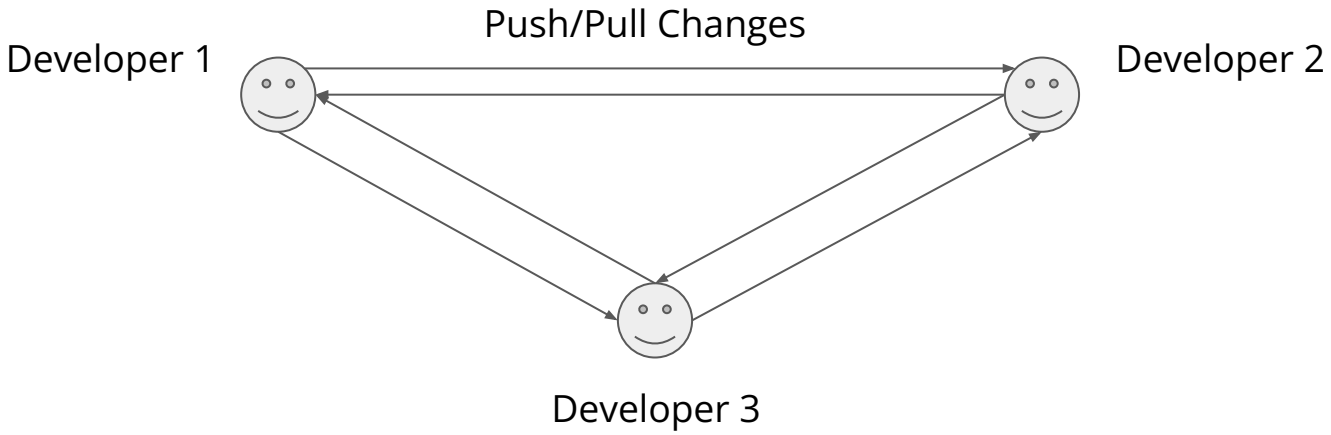
Learning a <u>ton</u> about git!

- Concepts & terminology: repository, remote, commit, branch
- Four phases of git
- Creating and merging branches, merge conflicts
- Various commands (`clone`, `add`, `commit`, `status`, `log`, `push`, `...`)

Recall: distributed version control

Git is a “distributed” version control system:
everyone working on the project has a <u>complete</u> version of the repository.

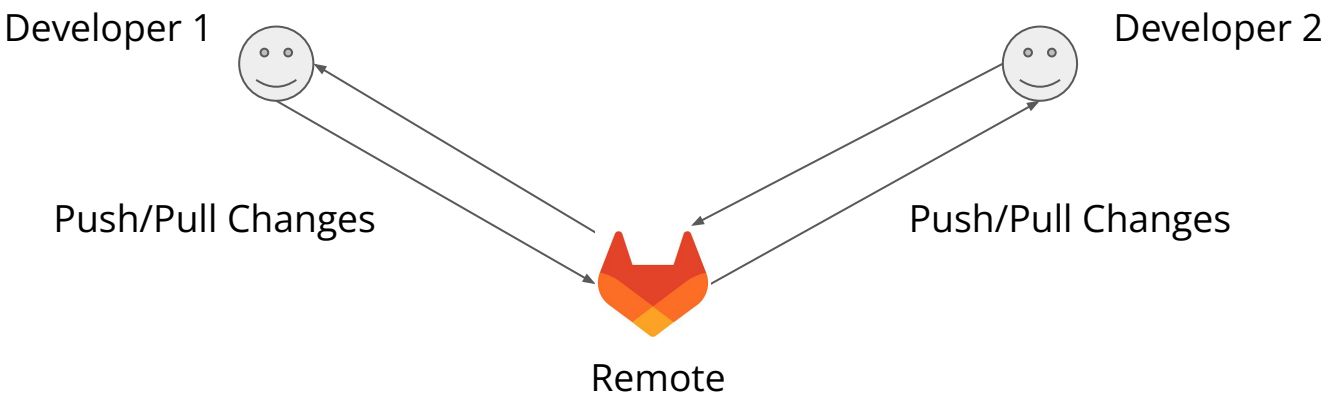
They communicate changes by sending them to each other.



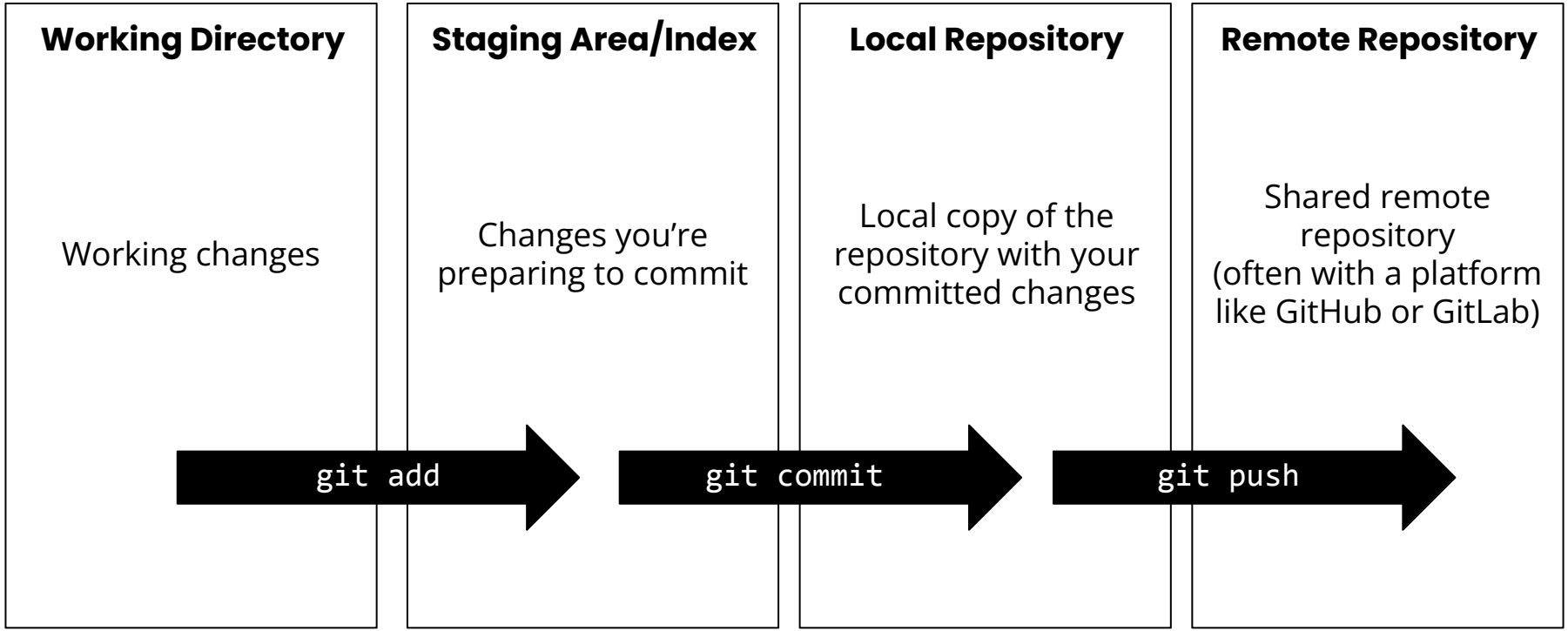
Recall: remote repositories

However, there is typically a special **remote** repository—a “central” source.

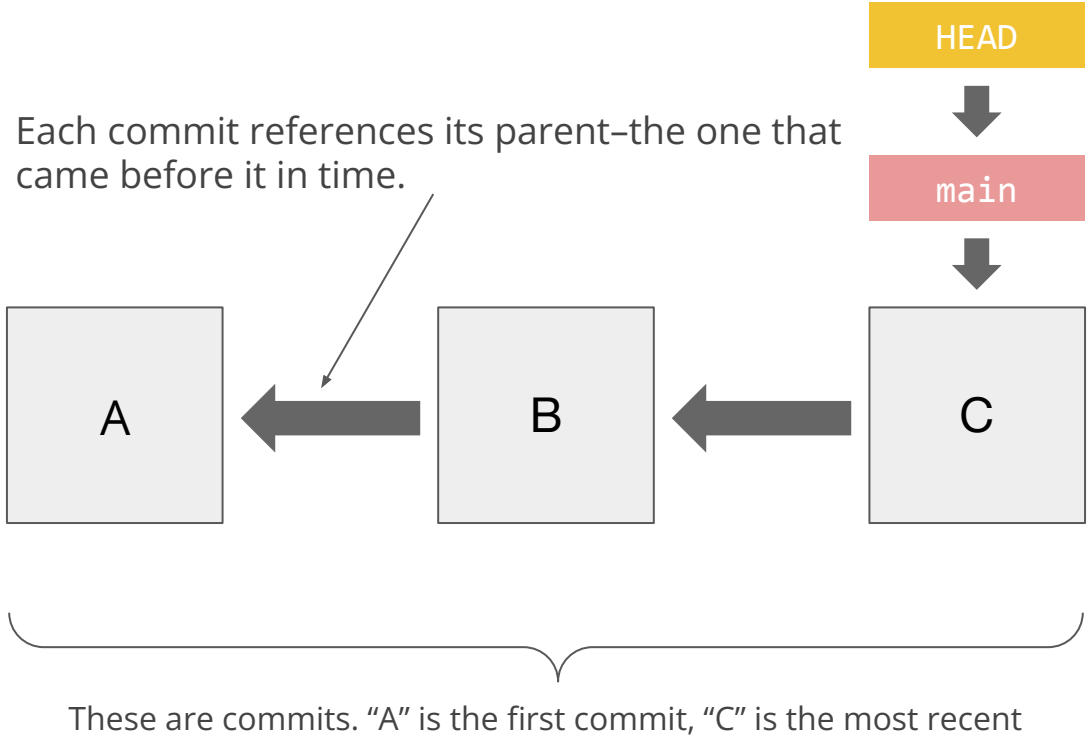
Remote is often stored on platforms like GitHub or GitLab;
everybody periodically syncs their changes with remote.



Recall: Git in Four Phases



Recall: commits as Linked Lists

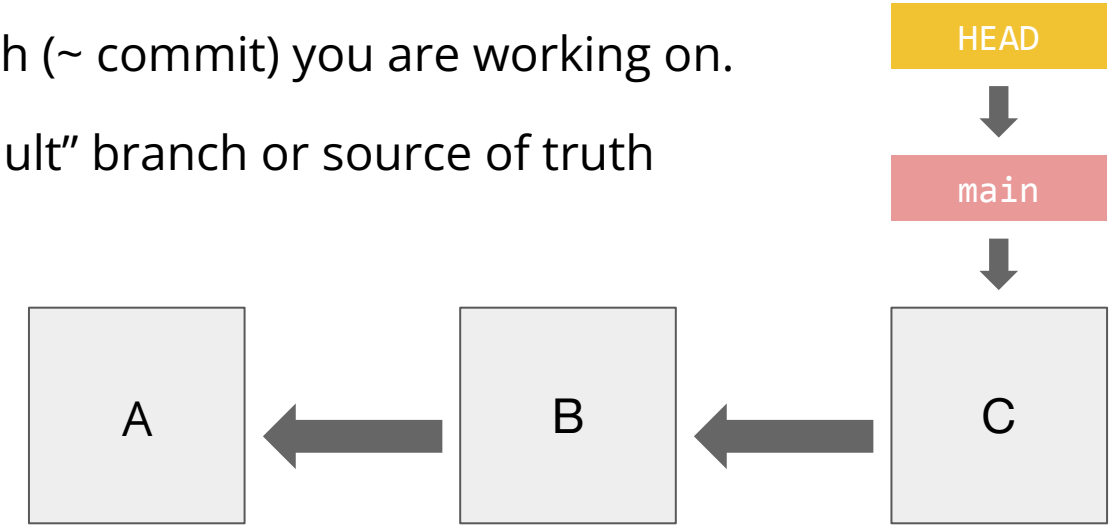


Recall: branches as pointers (or references)

A **branch** in git is a pointer/reference to a specific commit.

`HEAD`: the current branch (~ commit) you are working on.

`main`: typically the “default” branch or source of truth

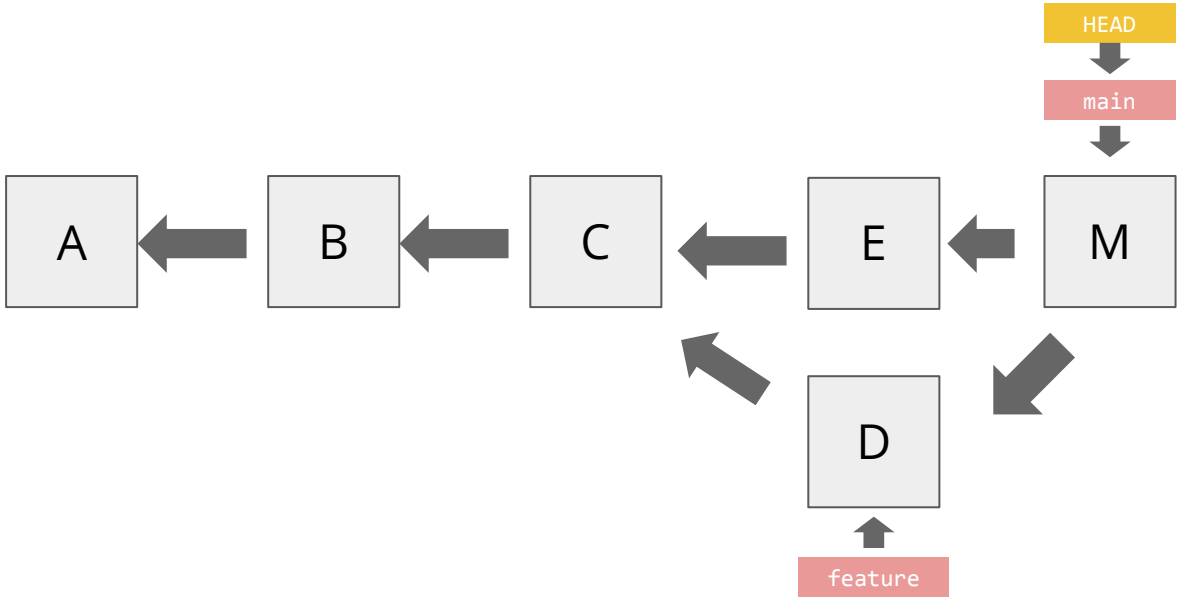


(to see this locally, run `git log --graph --oneline`)

Recall: creating & merging branches

Commands that have been run:

```
$ git branch feature
$ git checkout feature
$ echo "D" >> file.txt
$ git add file.txt
$ git commit -m "D"
$ git checkout main
$ echo "E" >> file.txt
$ git add file.txt
$ git commit -m "E"
$ git merge feature
```



Think, Pair, Share – Duplicitous Docs (DQ3)

One popular “version control” that you’ve probably used is Google Drive/Docs. How is this similar to git? How is it different?

live demo

(Ruth deploys the course website!)

Discussion Questions

Let’s work on some discussion questions :)

Suggested order:

1. quick conceptual review: Q1 & Q2
2. “HW4-lite”: Q4
3. extra time? Q5+

HW4: Getting Setup

We've made a repo for you for HW4!

- check your email and/or log in to GitLab to verify
- let us know ASAP if you don't see a repo

Can confirm that this repo "works" by doing up to Task 1 Problem 1 on HW4.

HW4: Getting Setup

I say again:

Let us know <u>ASAP</u> if you don’t see a repo!

HW4 Tips

Generally, this homework is a bit more challenging/less forgiving, especially since “undoing” your work is hard (but not impossible!)

- test in pieces!! (task-by-task)
- if comfortable: use `git revert`
- if really stuck, please reach out for help!
  - Happy to talk through commands
  - (We can also “reset” your repository for you, if you really would like so!)

rest: pre-lecture slides (what you saw in the video)

Never Have I Ever – Individual

When working on code by yourself, have you:

- accidentally broke your code, and can't revert back to the last working version?
- delete a super important file by accident
- broken your computer, losing your files
- when writing a paper, save each version as `final_paper.doc`, `final_paper2.doc`, `final_paper_actually_this_time.doc`

Never Have I Ever – Teams

When working on code with others, have you:

- had to figure out who keeps the most up-to-date version of the code?
- accidentally overwrote someone else's changes
- accidentally lose your own changes
- struggle to combine multiple people's changes
- struggled to find the last person who edited a specific file (or fixed a bug)
- and, deal with everything from the previous slide?

You Need Version Control!

Version control: software that keeps track of changes to a set of files.

You've seen version control in...

- Text editors that allow you to undo (Ctrl/Cmd + Z) and redo
- Google Docs / Sheets / Slides and its "see revision history" feature
- Non-text software like image and video editors

Git (and friends)

In this class, we'll learn about a version control system called **git**, the de facto standard for software development version control (and is used widely by industry, academia, etc.)

But, lots of people use version control outside of CS folks, including:

- architects, engineers, and artists making 3D assets (e.g. video games)
- musicians and sound engineers storing different versions of songs
- lawyers keeping track of contract changes over time

Repositories (1/2)

A **repository** (or **repo**) in git is a location that stores a copy of all files.

What should be in a repo?

- rule of thumb: everything you need to build the project from source
- source code (e.g. `.java`, `.c`, `.py` files)
- build and configuration files (e.g. `Makefile`, `build.xml`, `package.json`)
- assets (e.g. images) & documentation

Repositories (2/2)

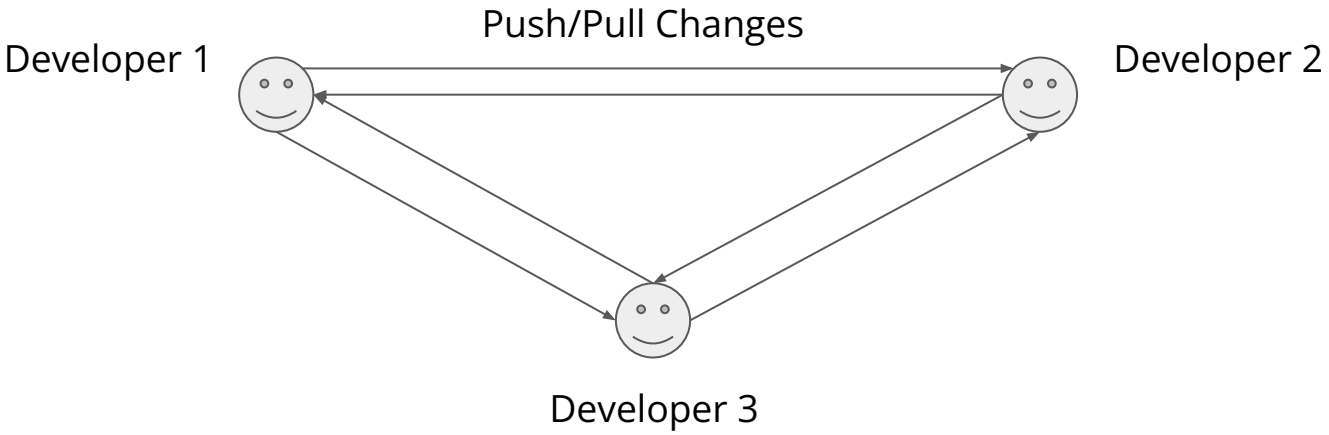
What should **<u>not</u>** be inside of a repository?

- Rule of thumb: things you can build *from* a source
- For example:
  - object files (e.g. `.class` or `.o` files)
  - executables (e.g. `.exe` or `.app` files)
  - sometimes: libraries & dependencies
    - often depends on your programming language, build system, and team preference

Repositories in Git

Git is a “distributed” version control system:
everyone working on the project has a complete version of the repository.

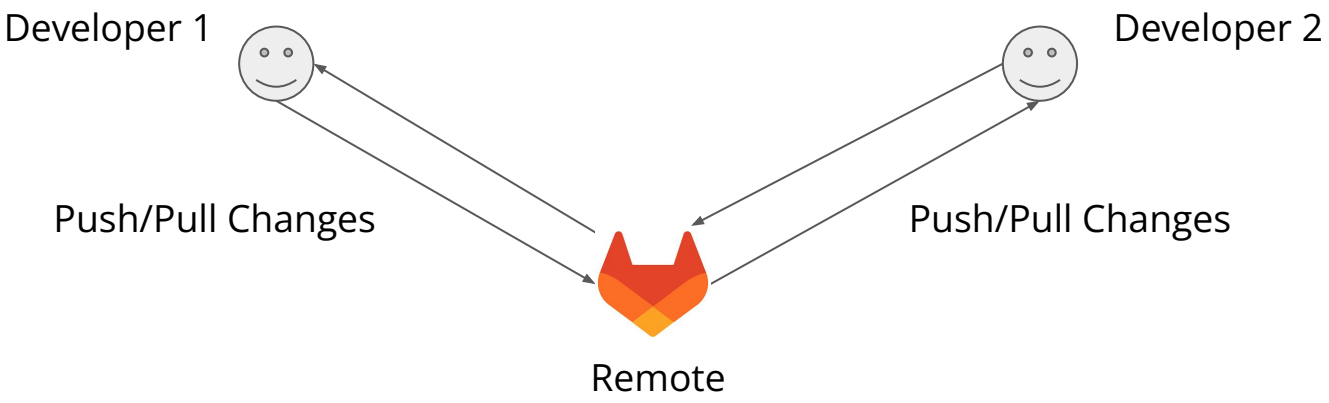
They communicate changes by sending them to each other.



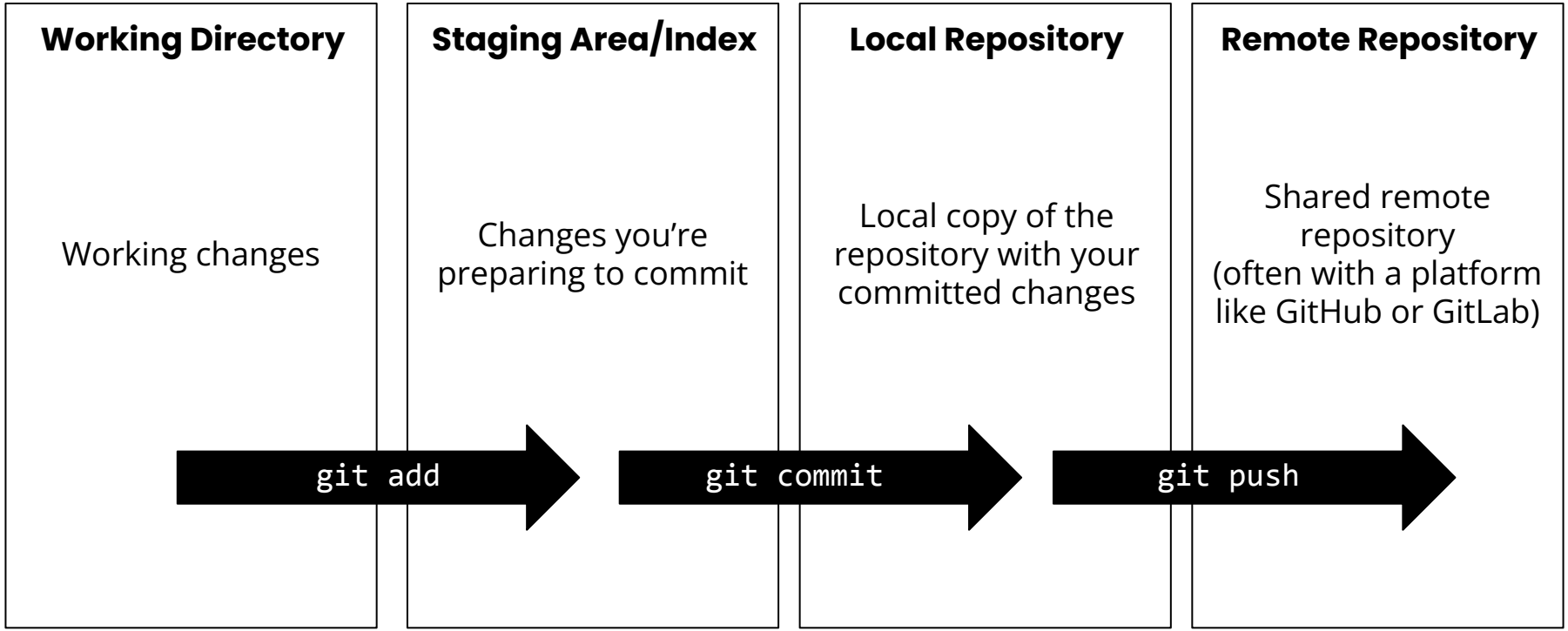
Remote Repositories

However, there is typically a special **remote** repository – a “central” source.

Remote is often stored on platforms like GitHub or GitLab;
everybody periodically syncs their changes with remote.



Git in Four Phases



🤔 Pause and Think (Getting Git)

Suppose a run of `git status` showed the following:

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
      modified:   file1.txt
      modified:   file2.txt
```

Based on this, which git phase are the changes to `file1.txt` and `file2.txt` currently in?

1. Working Directory
2. Staging Area
3. Local Repository
4. Remote Repository

🤔 Pause and Think (Ginormous Git)

Suppose a run of `git status` showed the following:

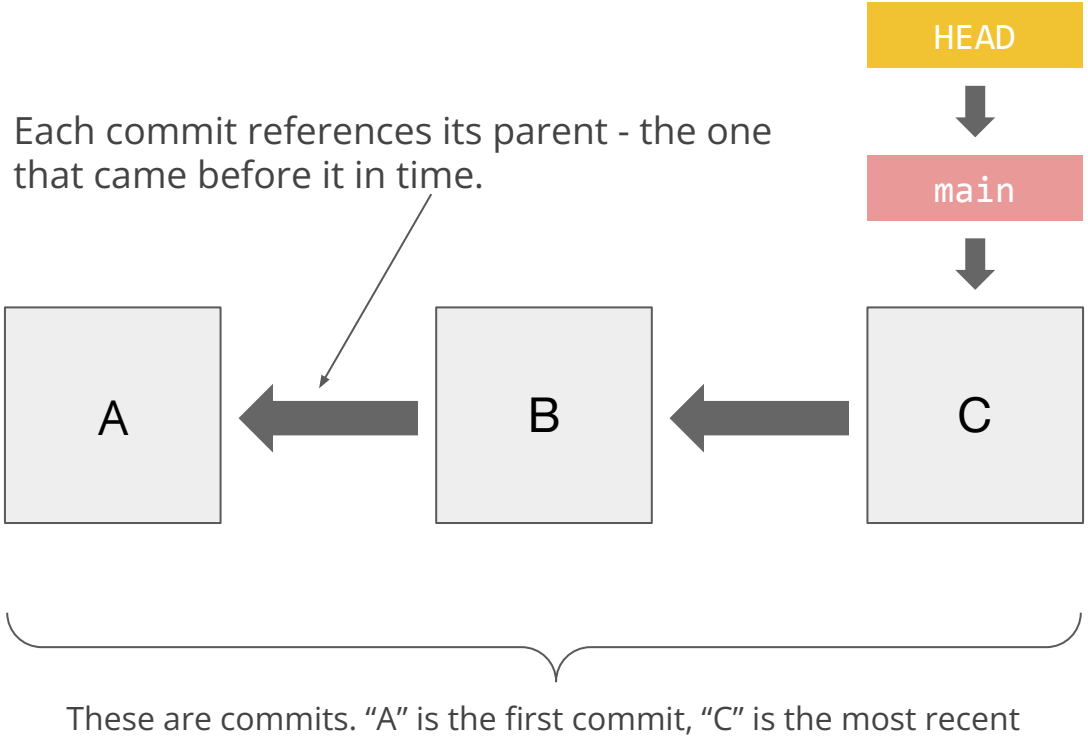
```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
      modified:   file1.txt
      modified:   file2.txt
```

Suppose we want to run a sequence of commands that:

1. includes the changes to file1.txt in a single commit
2. then, includes the changes to file2.txt in another commit
3. finally, reverts the change to *just* `file1.txt`

What commands would we run?

Commits as Linked Lists

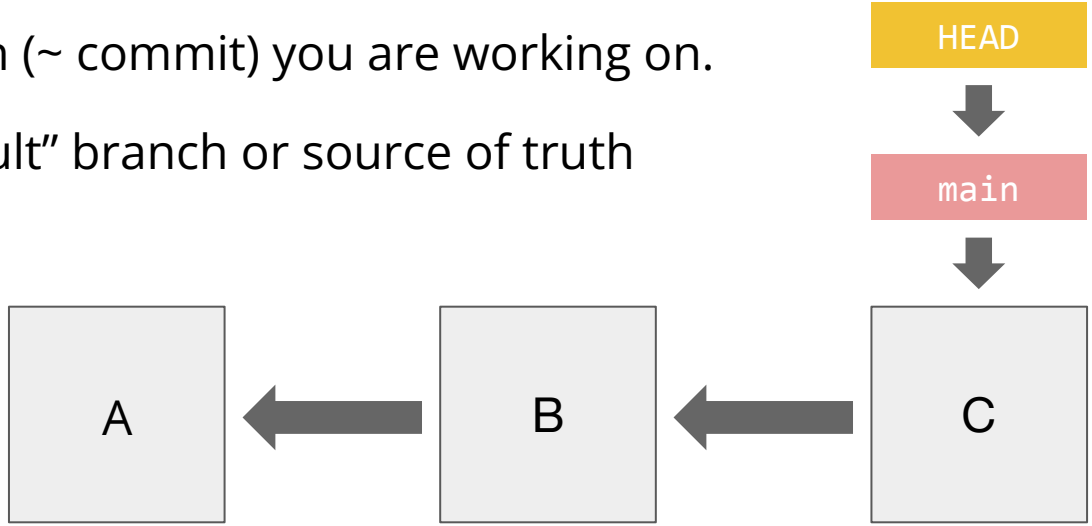


Branches as pointers (or references)

A **branch** in git is a pointer/reference to a specific commit.

`HEAD`: the current branch (~ commit) you are working on.

`main`: typically the “default” branch or source of truth



(to see this locally, run `git log --graph --oneline`)

Why branches?

So far, we've done all our work on the `main` branch.

However, you typically work on a separate branch:

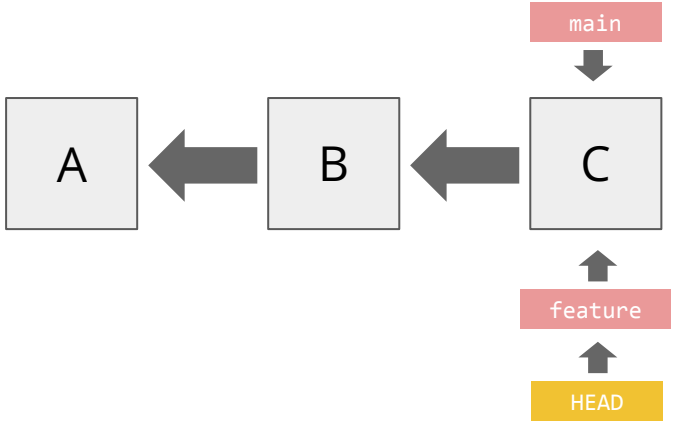
- main is the single source of truth – it should be stable (e.g. compile)
  - can't share or collaborate on in-progress work, since that would break the rule
  - also hard to work on two (or more) unrelated features at once
- instead, you'll do your work on a separate branch and "merge" it in
  - we'll ease you in to this process in HW4 & HW5!
  - for this video, we'll slightly break this rule – mostly for demonstration purposes

Note: main used to be called "master", though conventions have changed recently. You may see both names as you browse documentation & repos.

Branching (1/4)

Commands that have been run:

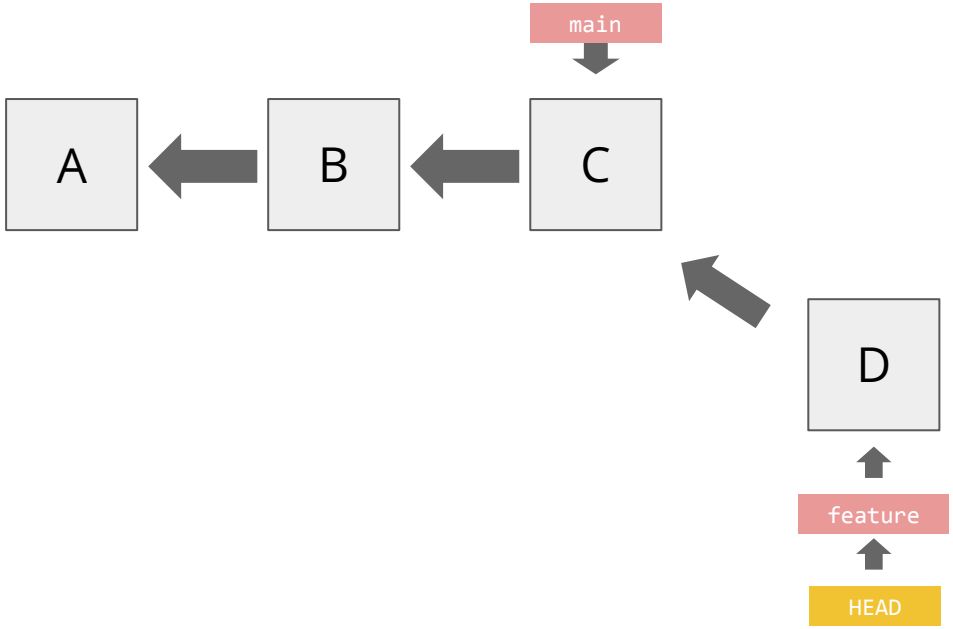
```
$ git branch feature
$ git checkout feature
```



Branching (2/4)

Commands that have been run:

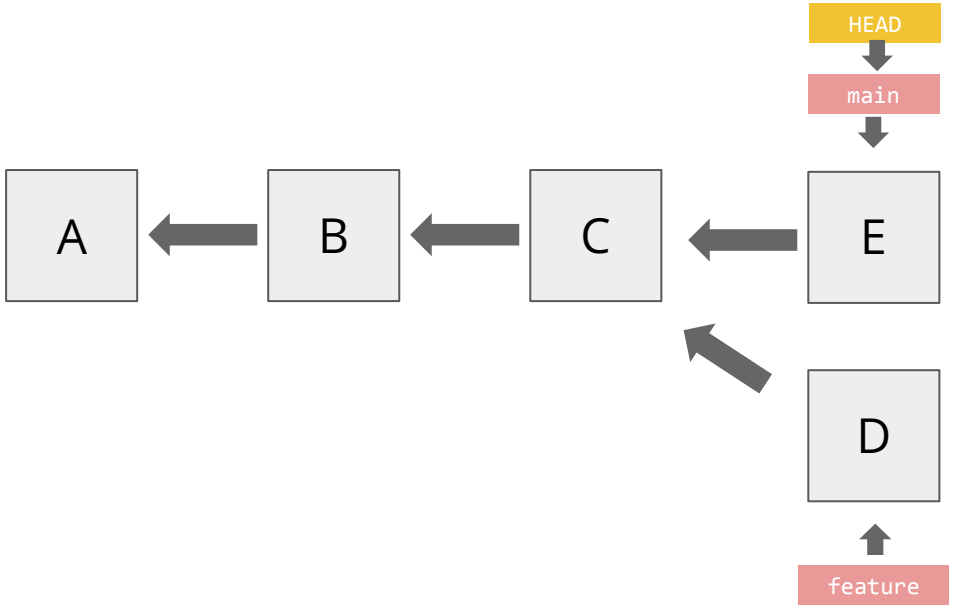
```
$ git branch feature
$ git checkout feature
$ echo "D" >> file.txt
$ git add file.txt
$ git commit -m "D"
```



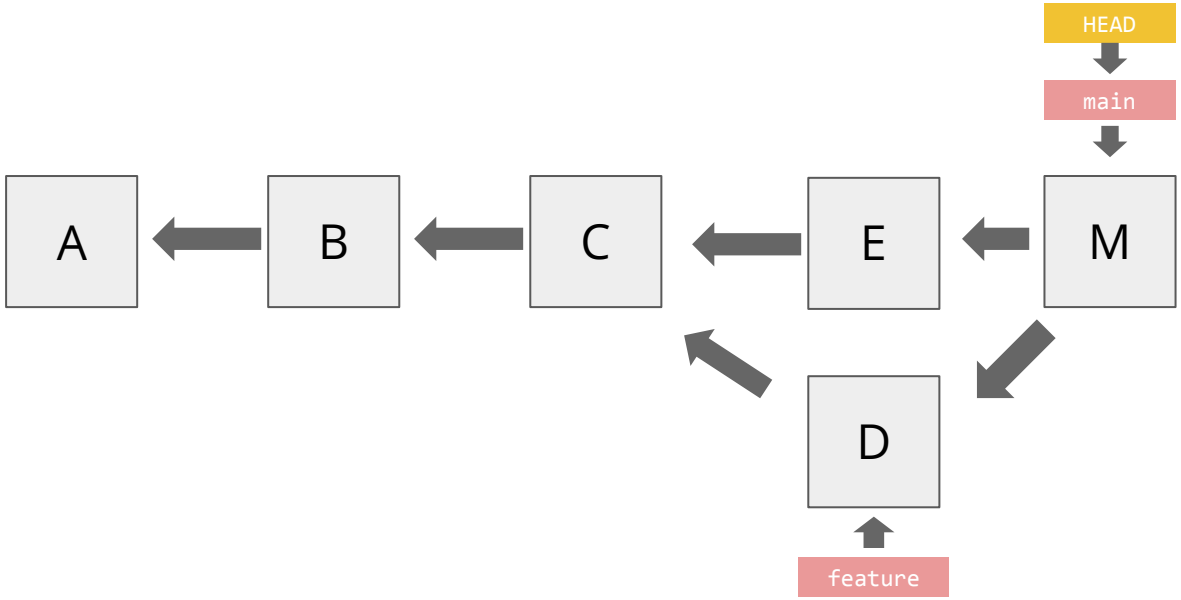
Branching (3/4)

Commands that have been run:

```
$ git branch feature
$ git checkout feature
$ echo "D" >> file.txt
$ git add file.txt
$ git commit -m "D"
$ git checkout main
$ echo "E" >> file.txt
$ git add file.txt
$ git commit -m "E"
```



Branching (4/4)

Commands that have been run:

```
$ git branch feature
$ git checkout feature
$ echo "D" >> file.txt
$ git add file.txt
$ git commit -m "D"
$ git checkout main
$ echo "E" >> file.txt
$ git add file.txt
$ git commit -m "E"
$ git merge feature
```